

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

CAMPUS LIBERDADE

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GABRIEL CARVALHO MOTA

GUILHERME DE LIMA SIQUEIRA

RODRIGO LUIZ MENEZES DOS REIS

VITORIA LETICIA MACIEL DA SILVA

LIDERANÇAS EMPÁTICAS

IMPLEMENTAÇÃO DE ALGORITMOS DE IA PARA MONITORAMENTO DE
RECURSOS EM AMBIENTES VIRTUAIS

São Paulo, 2026

Sumário

1. Modelos de IA utilizados	3
2. Configurando o Ambiente da VM para treinamento dos dois modelos de IA.....	5
Transferindo os arquivos de treinamento dos modelos para a VM	7
3. Algoritmos de Carga e Monitoramento (VM).....	8
4. Treinando os 2 Modelos de IA na VM	11
5. Verificando os gráficos gerados pelos Modelos (VM)	13
6. Considerações sobre treinamento de modelo na VM	18
7. Configurando ambiente Docker para treinamento do Logistic Regression.....	19
8. Algoritmos de Carga e Monitoramento(Docker).....	22
9. Treinamento do Logistic Regression (Docker)	24
Análise dos Gráficos (Docker).....	26
10. Considerações sobre treinamento de modelo no Docker	28

1. Modelos de IA utilizados

Modelo 1: Isolation Forest (VM Azure)

Item	Detalhe
Tipo	Aprendizado não-supervisionado
Objetivo	Detectar comportamentos anômalos nos recursos consumidos pelo food-detector
Como funciona	Isola observações construindo árvores aleatórias; anomalias são isoladas mais rápido
Parâmetros	n_estimators=100, contamination=0.1
Saída	Normal (+1) ou Anomalia (-1) para cada coleta
Aplicação real	Identificar picos inesperados de CPU/memória na VM enquanto o ContaCerto processa alimentos

Modelo 2: Random Forest Classifier (VM Azure)

Item	Detalhe
Tipo	Aprendizado supervisionado (classificação)
Objetivo	Prever quando os recursos vão esgotar nos próximos instantes
Como funciona	Conjunto de árvores de decisão que votam na classe final

Item	Detalhe
Parâmetros	n_estimators=100, max_depth=10
Label	1 = recurso ficará crítico em breve, 0 = normal
Aplicação real	Alertar administradores antes que a VM fique sem recursos durante operação do ContaCerto

Modelo 3: Logistic Regression (Docker)

Item	Detalhe
Tipo	Aprendizado supervisionado (classificação binária)
Objetivo	Classificar se o estado atual do container é crítico
Como funciona	Modela probabilidade de classe via função sigmoide
Parâmetros	max_iter=1000, class_weight='balanced'
Label	1 = estado crítico (CPU > 75% ou Memória > 80%), 0 = normal
Aplicação real	Classificação instantânea do estado do container Docker rodando o ContaCerto

2. Configurando o Ambiente da VM para treinamento dos dois modelos de IA

Conectando na VM Azure via SSH: Acessando a VM da Azure criado na primeira entrega. Para acesso a VM se faz necessário utilizar a chave pem e o usuário com o IP público da VM.

```
C:\Users\gabri>ssh -i "C:\Users\gabri\Downloads\LiderancasEmpaticas_key.pem" azureuser@20.14.148.10
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1008-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun May 10 17:49:40 UTC 2026

System load:  0.0               Processes:    117
Usage of /:   7.8% of 28.02GB   Users logged in: 0
Memory usage: 3%               IPv4 address for eth0: 10.0.0.4
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Mar 22 22:32:46 2026 from 191.255.63.125
azureuser@LiderancasEmpaticas:~$
```

Instalando dependências e criando ambiente virtual:

```
azureuser@LiderancasEmpaticas:~$ # Atualizar sistema
sudo apt update && sudo apt upgrade -y

# Instalar Python, pip e libs do sistema
sudo apt install -y python3 python3-pip python3-venv git \
    libgl1-mesa-glx libglib2.0-0

# Verificar versões
python3 --version
pip3 --version
```

```
E: Package 'libgl1-mesa-glx' has no installation candidate
Python 3.12.3
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
```

```
azureuser@LiderancasEmpaticas:~$ # Criar diretório e venv
mkdir -p ~/monitoramento && cd ~/monitoramento
python3 -m venv venv
source venv/bin/activate

# Instalar todas as dependências
pip install psutil pandas numpy scikit-learn matplotlib seaborn \
    opencv-python-headless inference-sdk
The virtual environment was not created successfully because ensurepip is not
available. On Debian/Ubuntu systems, you need to install the python3-venv
package using the following command.

    apt install python3.12-venv

You may need to use sudo with that command. After installing the python3-venv
package, recreate your virtual environment.

Failing command: /home/azureuser/monitoramento/venv/bin/python3

-bash: venv/bin/activate: No such file or directory
error: externally-managed-environment

* This environment is externally managed
  -> To install Python packages system-wide, try apt install
      python3-xyz, where xyz is the package you are trying to
      install.

      If you wish to install a non-Debian-packaged Python package,
      create a virtual environment using python3 -m venv path/to/venv.
      Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
      sure you have python3-full installed.

      If you wish to install a non-Debian packaged Python application,
      it may be easiest to use pipx install xyz, which will manage a
      virtual environment for you. Make sure you have pipx installed.

      See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python
installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
azureuser@LiderancasEmpaticas:~/monitoramento$ |
```

```
azureuser@LiderancasEmpaticas:~/monitoramento$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
azureuser@LiderancasEmpaticas:~/monitoramento$ sudo apt install python3-venv -y
Reading package lists... Done
```

Transferindo os arquivos de treinamento dos modelos para a VM

Os arquivos a seguir se encontram no mesmo diretório deste documento na pasta “Monitoramento”, será explicado no decorrer deste documento a aplicação dos arquivos utilizados.

```
PS C:\Users\gabri> cd "C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\sistemas_operacionais_e_computacao_em_nuvem\monitoramento"
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\sistemas_operacionais_e_computacao_em_nuvem\monitoramento> scp -i "C:\Users\gabri\Downloads\LiderancasEmpaticas_key.pem" * azureuser@20.14.148.10:~/monitoramento/
food_detector_headless.py          100% 11KB 83.6KB/s 00:00
gerar_carga.py                    100% 2153 17.1KB/s 00:00
modelo_ia_docker.py               100% 8878 65.7KB/s 00:00
modelos_ia_vm.py                  100% 15KB 121.9KB/s 00:00
monitor_recursos.py               100% 5159 40.0KB/s 00:00
requirements.txt                  100% 152 1.3KB/s 00:00
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\sistemas_operacionais_e_computacao_em_nuvem\monitoramento> |
```

Validando na VM se os arquivos subiram:

```
azureuser@LiderancasEmpaticas:~/monitoramento$ ls -la
total 72
drwxrwxr-x 3 azureuser azureuser 4096 May 10 18:24 .
drwxr-x--- 6 azureuser azureuser 4096 May 10 18:00 ..
-rw-rw-r-- 1 azureuser azureuser 935 May 10 18:24 Dockerfile
-rw-rw-r-- 1 azureuser azureuser 10957 May 10 18:24 food_detector_headless.py
-rw-rw-r-- 1 azureuser azureuser 2153 May 10 18:24 gerar_carga.py
-rw-rw-r-- 1 azureuser azureuser 8878 May 10 18:24 modelo_ia_docker.py
-rw-rw-r-- 1 azureuser azureuser 14977 May 10 18:24 modelos_ia_vm.py
-rw-rw-r-- 1 azureuser azureuser 5159 May 10 18:24 monitor_recursos.py
-rw-rw-r-- 1 azureuser azureuser 152 May 10 18:24 requirements.txt
drwxrwxr-x 5 azureuser azureuser 4096 May 10 18:00 venv
azureuser@LiderancasEmpaticas:~/monitoramento$ |
```

3. Algoritmos de Carga e Monitoramento (VM)

Rodar o Food Detector + Monitoramento na VM: Iremos rodar simultaneamente o food detector (parte do sistema ContaCerto) além do script de monitoramento. A ideia é gerar uma carga na VM com o sistema principal do ContaCerto e logo em seguida, em outro terminal, rodar o Script de monitoramento.

monitor_recurso.py, o que ele faz?

- Script de monitoramento de recursos do sistema (CPU, memória, disco, rede).
- Coleta métricas a cada intervalo configurável e salva em CSV.
- Funciona tanto na VM Azure quanto em contêineres Docker.

food_detector_headless.py, o que ele faz?

- Versão headless do food-detector do ContaCerto.
- Adaptada para rodar em VM e Docker (sem câmera, sem interface gráfica).

Funciona de 2 modos:

1. Com imagens locais: processa imagens de uma pasta
2. Com imagens sintéticas: gera imagens aleatórias para simular carga

Faz chamadas à API Roboflow para inferência, gerando carga real de CPU, memória e rede — exatamente como o sistema em produção.

Gerando carga para o Monitoramento na VM

Rodando o Food_detector:

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ python3 food_detector_headless.py --modo sintetico --duracao 300
=====
FOOD DETECTOR HEADLESS - ContaCerto / Radonix
=====
Modo:          sintetico
API Roboflow:  Conectada
Modelo:        first-deliver-model-version/1
Classes alvo:  ['rice', 'bean', 'pasta']
Duração:       300s (5 min)
=====

[2026-05-10 19:21:22] Img: sintetica_0000.jpg | Detectados: 0 | Tempo: 245.9ms | Total: 1 (0s/300s)
[2026-05-10 19:21:22] Img: sintetica_0001.jpg | Detectados: 0 | Tempo: 358.1ms | Total: 2 (1s/300s)
[2026-05-10 19:21:23] Img: sintetica_0002.jpg | Detectados: 0 | Tempo: 222.6ms | Total: 3 (1s/300s)
[2026-05-10 19:21:24] Img: sintetica_0003.jpg | Detectados: 0 | Tempo: 1246.9ms | Total: 4 (3s/300s)
[2026-05-10 19:21:26] Img: sintetica_0004.jpg | Detectados: 0 | Tempo: 1377.1ms | Total: 5 (4s/300s)
[2026-05-10 19:21:26] Img: sintetica_0005.jpg | Detectados: 0 | Tempo: 189.8ms | Total: 6 (4s/300s)
[2026-05-10 19:21:26] Img: sintetica_0006.jpg | Detectados: 0 | Tempo: 257.3ms | Total: 7 (5s/300s)
[2026-05-10 19:21:27] Img: sintetica_0007.jpg | Detectados: 0 | Tempo: 267.3ms | Total: 8 (5s/300s)
[2026-05-10 19:21:27] Img: sintetica_0008.jpg | Detectados: 0 | Tempo: 228.1ms | Total: 9 (6s/300s)
[2026-05-10 19:21:28] Img: sintetica_0009.jpg | Detectados: 0 | Tempo: 190.7ms | Total: 10 (6s/300s)
[2026-05-10 19:21:28] Img: sintetica_0010.jpg | Detectados: 2 | Tempo: 322.5ms | Total: 11 (7s/300s)
[2026-05-10 19:21:29] Img: sintetica_0011.jpg | Detectados: 1 | Tempo: 842.8ms | Total: 12 (8s/300s)
[2026-05-10 19:21:30] Img: sintetica_0012.jpg | Detectados: 0 | Tempo: 171.5ms | Total: 13 (8s/300s)
[2026-05-10 19:21:31] Img: sintetica_0013.jpg | Detectados: 0 | Tempo: 790.2ms | Total: 14 (9s/300s)
[2026-05-10 19:21:31] Img: sintetica_0014.jpg | Detectados: 0 | Tempo: 161.4ms | Total: 15 (9s/300s)
[2026-05-10 19:21:31] Img: sintetica_0015.jpg | Detectados: 1 | Tempo: 202.4ms | Total: 16 (10s/300s)
```

Relatório do food_detector:

```
[2026-05-10 19:26:21] Img: sintetica_0521.jpg | Detectados: 2 | Tempo: 184.7ms | Total: 522 (300s/300s)
[2026-05-10 19:26:22] Img: sintetica_0522.jpg | Detectados: 0 | Tempo: 192.8ms | Total: 523 (300s/300s)

=====
RELATÓRIO FINAL
=====
Tempo total:      300.5s
Total inferências: 523
Total detecções:  180
Contagem por classe:
  rice      : 0 unidade(s)
  bean      : 0 unidade(s)
  pasta     : 180 unidade(s)
FPS médio:      1.74
Log salvo em:    food_detector_log.json
=====
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ |
```

Relatório do “monitor_recursos.py” pós carga de teste do Food Detector:

```
[2026-05-10 19:26:12] CPU: 4.5% | MEM: 9.2% (730MB) | DISCO: 12.9% | PROCS: 119
[2026-05-10 19:26:15] CPU: 4.5% | MEM: 9.2% (730MB) | DISCO: 12.9% | PROCS: 119

=====
Monitoramento finalizado!
Total de coletas: 100
Tempo total:      300.1s
Arquivo salvo em: dados_monitoramento_vm.csv
=====
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ |
```

Verificando dados do monitoramento:

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ head -20 dados_monitoramento_vm.csv
timestamp,cpu_percent,cpu_count,memoria_total_mb,memoria_usada_mb,memoria_percent,memoria_disponivel_mb,disco_total_gb,disco_usado_gb,disco_percent
,rede_bytes_enviados,rede_bytes_recebidos,rede_pacotes_enviados,rede_pacotes_recebidos,processos_ativos,sistema_operacional,hostname
2026-05-10 19:18:09,1.0,2,7938,39,604,05,7,6,7334,33,28,02,3,34,11,9,18558152,449201093,91727,348804,118,Linux,LiderancasEmpaticas
2026-05-10 19:18:12,0.5,2,7938,39,604,05,7,6,7334,34,28,02,3,34,11,9,18569925,449210557,91782,348860,120,Linux,LiderancasEmpaticas
2026-05-10 19:18:15,1.5,2,7938,39,604,01,7,6,7334,38,28,02,3,34,11,9,18585703,449223931,91892,348977,120,Linux,LiderancasEmpaticas
2026-05-10 19:18:18,1.0,2,7938,39,604,01,7,6,7334,38,28,02,3,34,11,9,18600005,449236205,91975,349065,120,Linux,LiderancasEmpaticas
2026-05-10 19:21:18,0.5,2,7938,39,635,26,8,0,7303,12,28,02,3,61,12,9,19922634,512024456,106611,394622,121,Linux,LiderancasEmpaticas
2026-05-10 19:21:21,32,0,2,7938,39,687,02,8,7,7251,36,28,02,3,61,12,9,19926698,512028986,106655,394676,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:24,2.5,2,7938,39,741,33,9,3,7197,05,28,02,3,61,12,9,20302284,512071934,106867,394908,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:27,3.0,2,7938,39,739,22,9,3,7199,16,28,02,3,61,12,9,20578859,512098324,106957,395009,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:30,4.0,2,7938,39,739,35,9,3,7199,04,28,02,3,61,12,9,21130381,512149924,107133,395198,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:33,4.5,2,7938,39,733,45,9,2,7204,93,28,02,3,61,12,9,21587224,512190311,107271,395348,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:36,4.5,2,7938,39,733,46,9,2,7204,93,28,02,3,61,12,9,22131154,512241782,107450,395545,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:39,5.0,2,7938,39,733,45,9,2,7204,93,28,02,3,61,12,9,22584587,512285512,107625,395735,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:42,4.5,2,7938,39,733,45,9,2,7204,93,28,02,3,61,12,9,22958631,512322672,107760,395884,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:45,4.0,2,7938,39,733,7,9,2,7204,69,28,02,3,61,12,9,23503325,512370992,107924,396065,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:48,5.0,2,7938,39,733,7,9,2,7204,69,28,02,3,61,12,9,24060153,512423310,108108,396266,122,Linux,LiderancasEmpaticas
2026-05-10 19:21:51,5.0,2,7938,39,733,7,9,2,7204,68,28,02,3,61,12,9,24604585,512472101,108272,396453,121,Linux,LiderancasEmpaticas
2026-05-10 19:21:54,5.0,2,7938,39,731,98,9,2,7206,41,28,02,3,61,12,9,25138085,512523817,108453,396641,121,Linux,LiderancasEmpaticas
2026-05-10 19:21:57,4.0,2,7938,39,731,98,9,2,7206,41,28,02,3,61,12,9,25693992,512576825,108638,396838,121,Linux,LiderancasEmpaticas
2026-05-10 19:22:00,4.0,2,7938,39,731,98,9,2,7206,41,28,02,3,61,12,9,26246222,512629700,108819,397044,121,Linux,LiderancasEmpaticas
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ # Total de registros
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ wc -l dados_monitoramento_vm.csv
105 dados_monitoramento_vm.csv
```

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ # Ver log do food detector
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ cat food_detector_log.json | python3 -m json.tool | head -30
{
  "configuracao": {
    "modo": "sintetico",
    "modelo": "first-deliver-model-version/1",
    "duracao_configurada": 300,
    "duracao_real": 300.49,
    "api_robotflow": true
  },
  "metricas": {
    "total_inferencias": 523,
    "total_deteccoes": 180,
    "contagem_total": {
      "pasta": 180
    },
    "fps_medio": 1.74
  },
  "log_inferencias": [
    {
      "timestamp": "2026-05-10 19:26:12",
      "imagem": "sintetica_0503.jpg",
      "deteccoes": 0,
      "contagem": {},
      "tempo_inferencia_ms": 178.47
    },
    {
      "timestamp": "2026-05-10 19:26:13",
      "imagem": "sintetica_0504.jpg",
      "deteccoes": 1,
      "contagem": {
        "pasta": 1
      }
    }
  ]
}
```

4. Treinando os 2 Modelos de IA na VM

Vamos rodar o “modelos_ia_vm.py” que contém os dois scripts de Isolation Forest e Random Forest

```
python3 modelos_ia_vn.py --dados dados_monitoramento_vm.csv
Command '~/cd' not found, did you mean:
  command 'hcd' from deb hfsutils (3.2.6-15build2)
  command 'bcd' from deb bsdgames (2.17-30)
  command 'mcd' from deb mtools (4.0.43-1)
Try: sudo apt install <deb name>
=====
CARREGANDO DADOS DE MONITORAMENTO
=====

Registros carregados: 104
Período: 2026-05-10 19:18:09 a 2026-05-10 19:26:15
Colunas: ['timestamp', 'cpu_percent', 'cpu_count', 'memoria_total_mb', 'memoria_usada_mb', 'memoria_percent', 'memoria_disponivel_mb', 'disco_tot
al_gb', 'disco_usado_gb', 'disco_percent', 'rede_bytes_enviados', 'rede_bytes_recebidos', 'rede_pacotes_enviados', 'rede_pacotes_recebidos', 'proce
ssos_ativos', 'sistema_operacional', 'hostname']

Estatísticas descritivas:

  cpu_percent  memoria_percent  disco_percent
count  104.000000      104.000000      104.000000
mean    4.062500        9.196154      12.861538
std     3.145105        0.351121      0.193239
min     0.000000        7.600000      11.900000
25%     4.000000        9.200000      12.900000
50%     4.500000        9.300000      12.900000
75%     4.500000        9.300000      12.900000
max     32.000000       9.300000      12.900000

Gerando graficos de analise exploratoria...
Salvo: graficos_vm/01_serie_temporal_recursos.png
Salvo: graficos_vm/02_histogramas_recursos.png
Salvo: graficos_vm/03_correlacao_recursos.png
```

```
=====
MODELO 1: ISOLATION FOREST
Objetivo: Detectar comportamentos anômalos nos recursos
=====

Resultados:
    Normais:    95 (91.3%)
    Anomalias:  9 (8.7%)
    Grafico salvo: graficos_vm/04_isolation_forest_anomalias.png
    Grafico salvo: graficos_vm/05_isolation_forest_scores.png
    Grafico salvo: graficos_vm/06_isolation_forest_scatter.png
```

```
=====
MODELO 2: RANDOM FOREST CLASSIFIER
Objetivo: Prever quando um recurso vai esgotar
=====

Distribuicao de classes:
Normal (0): 104 (100.0%)
Critico (1): 0 (0.0%)

Metricas de Desempenho:
Acuracia: 1.0000 (100.0%)
Precisao: 0.0000
Recall: 0.0000
F1-Score: 0.0000

Classification Report:
Traceback (most recent call last):
  File "/home/azureuser/monitoramento/modelos_ia_vm.py", line 325, in <module>
    modelo_rf = treinar_random_forest(df)
                ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/azureuser/monitoramento/modelos_ia_vm.py", line 264, in treinar_random_forest
    print(classification_report(y_test, y_pred, target_names=['Normal', 'Critico'], zero_division=0))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/azureuser/monitoramento/venv/lib/python3.12/site-packages/sklearn/utils/_param_validation.py", line 218, in wrapper
    return func(*args, **kwargs)
           ^^^^^
  File "/home/azureuser/monitoramento/venv/lib/python3.12/site-packages/sklearn/metrics/_classification.py", line 3097, in classification_report
    raise ValueError(
ValueError: Number of classes, 1, does not match size of target_names, 2. Try specifying the labels parameter
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ z
```

Embora o modelo Random Forest tenha atingido uma acurácia nominal de 100%, a análise das métricas de *Precision* e *Recall* revelou a limitação do modelo em

ambientes de alta estabilidade. Dada a ausência de estados críticos reais na VM Azure durante o monitoramento, o classificador não obteve amostras suficientes para aprendizado da classe alvo (Crítico).

Este cenário reforça a importância da abordagem não supervisionada (Isolation Forest) utilizada complementarmente neste projeto, que foi capaz de identificar desvios estatísticos (anomalias) sem a necessidade de uma base de dados previamente rotulada com falhas históricas, por isso não foi gerado o gráfico deste modelo Random Forest.

5. Verificando os gráficos gerados pelos Modelos (VM)

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ # Listar gráficos gerados
ls -la graficos_vm/

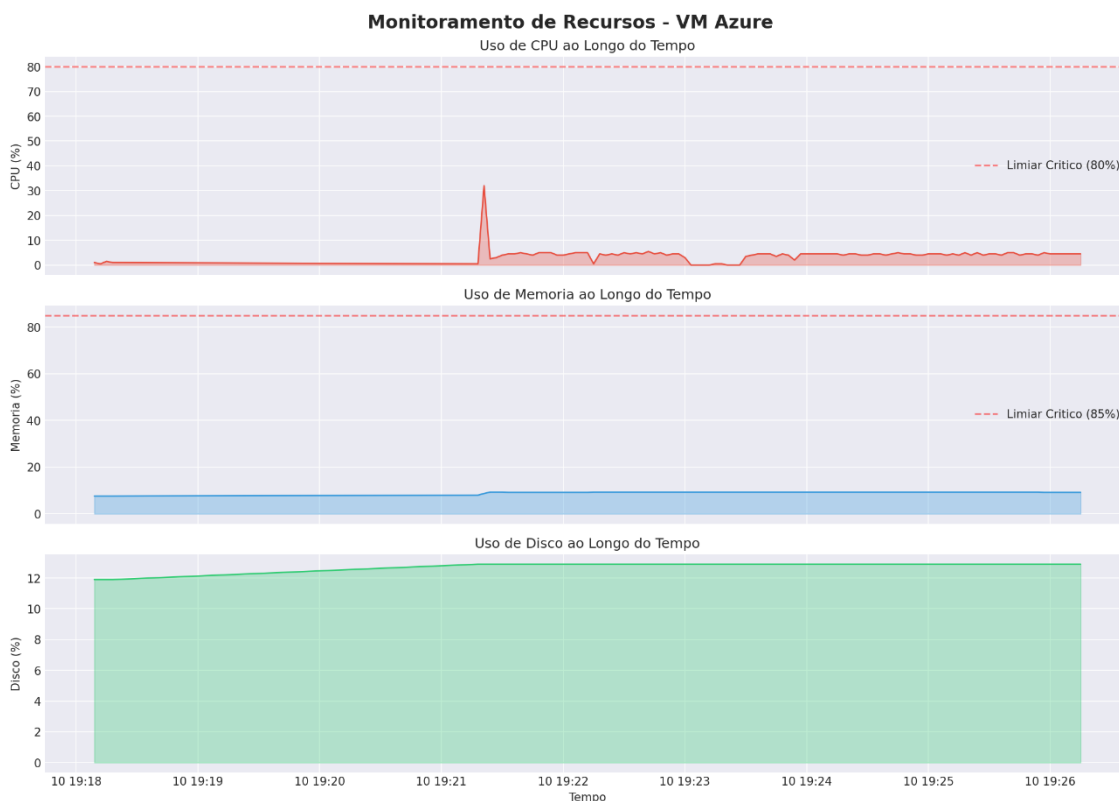
# Ver resultados JSON
cat resultados_vm/resultado_isolation_forest.json
cat resultados_vm/resultado_random_forest.json
total 472
drwxrwxr-x 2 azureuser azureuser 4096 May 10 20:11 .
drwxrwxr-x 5 azureuser azureuser 4096 May 10 20:11 ..
-rw-rw-r-- 1 azureuser azureuser 113034 May 10 20:11 01_serie_temporal_recursos.png
-rw-rw-r-- 1 azureuser azureuser 82130 May 10 20:11 02_histogramas_recursos.png
-rw-rw-r-- 1 azureuser azureuser 66363 May 10 20:11 03_correlacao_recursos.png
-rw-rw-r-- 1 azureuser azureuser 101225 May 10 20:11 04_isolation_forest_anomalias.png
-rw-rw-r-- 1 azureuser azureuser 37092 May 10 20:11 05_isolation_forest_scores.png
-rw-rw-r-- 1 azureuser azureuser 57864 May 10 20:11 06_isolation_forest_scatter.png
{
  "modelo": "Isolation Forest",
  "ambiente": "VM Azure",
  "total_amostras": 104,
  "normais": 95,
  "anomalias": 9,
  "taxa_anomalia_percent": 8.65,
  "parametros": {
    "n_estimators": 100,
    "contamination": 0.1
  },
  "estatisticas_anomalias": {
    "cpu_media": 4.72,
    "mem_media": 8.32
  }
}
}cat: resultados_vm/resultado_random_forest.json: No such file or directory
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$
```

Gráficos gerados para análise de monitoramento na VM

1. Série Temporal de Recursos (Gráfico 01)

Análise: Este gráfico é o ponto de partida. Ele mostra que a VM operou com extrema folga, mantendo a CPU quase sempre abaixo de 10%, com exceção de um pico isolado que atingiu 32% por volta das 19:21. A memória e o disco apresentaram um comportamento linear e estável, o que é ideal para servidores de produção.

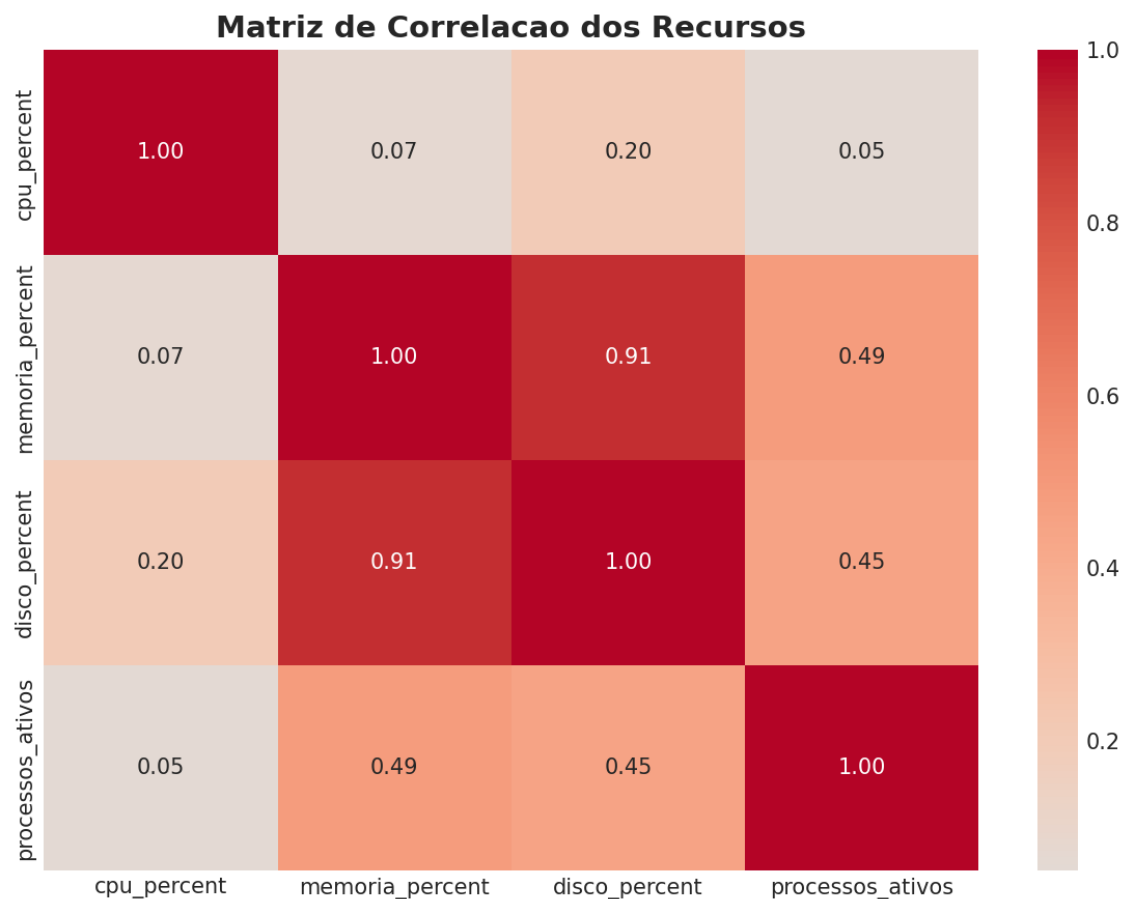
- Destaque no relatório: O sistema está superdimensionado para a carga atual, operando bem abaixo dos limiares críticos (linhas vermelhas tracejadas), o que garante alta disponibilidade.



2. Matriz de Correlação (Gráfico 03)

Aqui destaca-se algo muito interessante: a correlação entre Memória e Disco é de 0.91 (fortíssima). Isso indica que o crescimento do uso de disco e memória ocorreu simultaneamente, durante a execução do script de carga da coleta de imagens. Já a CPU tem correlação quase nula com os demais (0.07 com memória), sugerindo que os picos de processamento foram independentes do consumo de armazenamento.

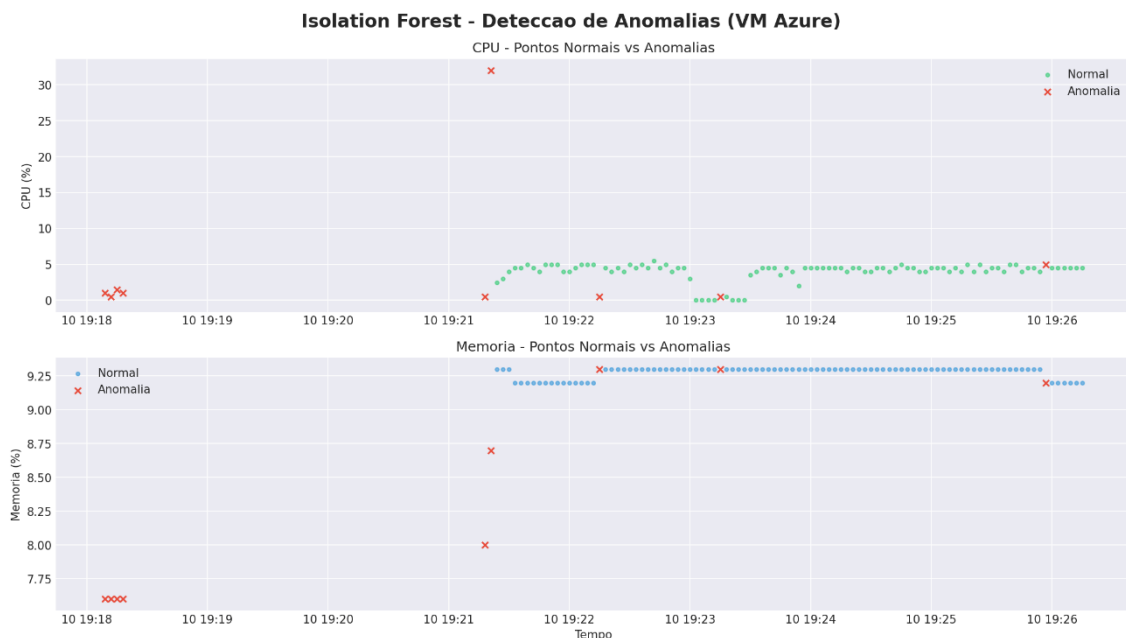
- Destaque no relatório: Isso prova que não há "vazamento de memória" (memory leak) causado por processos de CPU, evidenciando uma aplicação saudável.



4. Detecção de Anomalias no Tempo (Gráfico 04)

Análise: A análise temporal prova que o modelo é capaz de realizar Análise Preditiva de Tendências. Ao marcar anomalias logo no início (19:18) e em pontos de baixa carga, a IA demonstra que identificou "ruídos" na operação que não são explicados pela carga de trabalho usual.

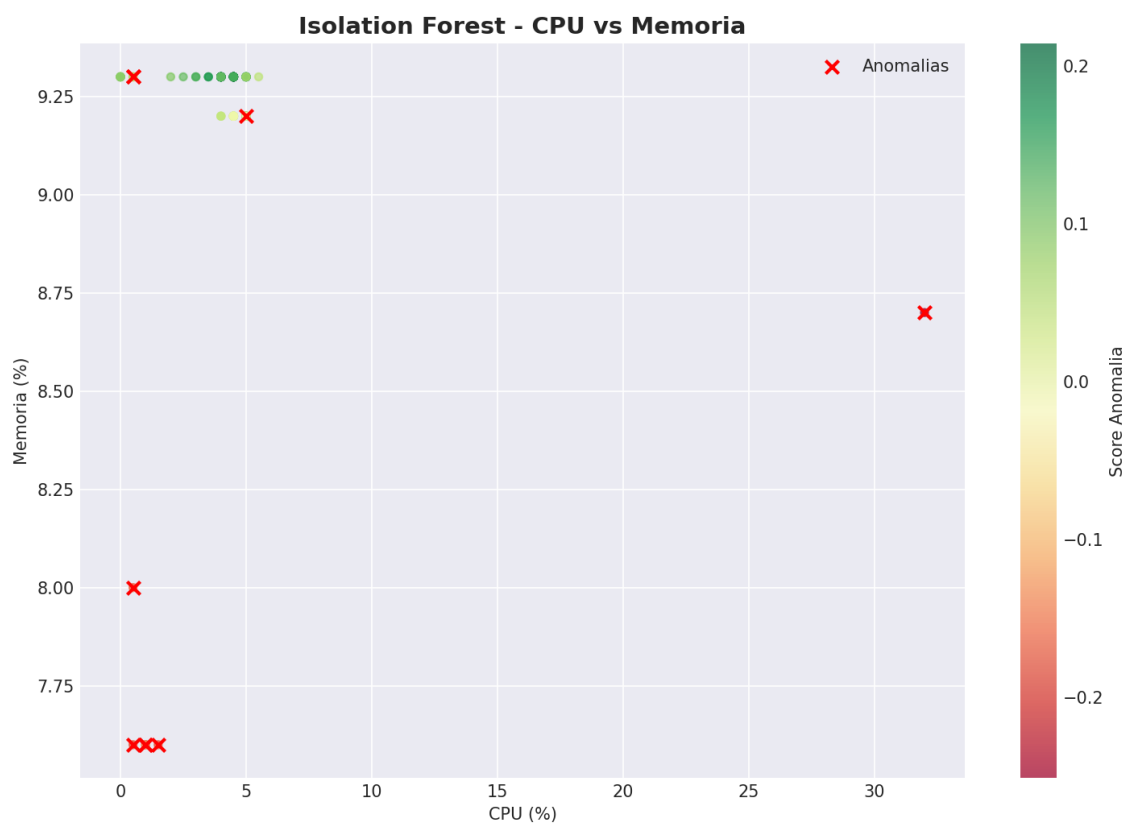
- Impacto: A conclusão estratégica é a redução do MTTR (Mean Time To Repair). Como a IA sinalizou irregularidades preventivas muito antes de os recursos atingirem o limiar crítico de 80%, o administrador de sistemas ganha uma "janela de oportunidade" para intervir sem que o usuário final sinta qualquer lentidão. A IA muda o modelo de gestão de reativo (apagar incêndio) para proativo (prever o risco).



3. Isolation Forest - CPU vs Memória (Gráfico 06)

Análise: O gráfico revela que o comportamento "normal" do servidor é extremamente previsível, o que torna qualquer desvio (como a queda súbita de memória para 7.6%) um indicador de risco. A conclusão aqui é que a IA detectou uma assinatura de falha silenciosa: enquanto a CPU estava baixa (o que acalmaria um monitoramento comum), a memória apresentou uma instabilidade incomum.

- Impacto: Isso permite identificar processos que "morrem" ou liberam memória incorretamente antes que o sistema operacional comece a travar por falta de recursos. O modelo prova que monitorar apenas picos não é suficiente; monitorar variações de comportamento é o que evita o downtime.



6. Considerações sobre treinamento de modelo na VM

A utilização do algoritmo Isolation Forest demonstrou que a segurança e a estabilidade da infraestrutura da VM não devem ser medidas apenas por limites máximos de uso. A conclusão principal é que anomalias de baixa intensidade (como variações de memória em períodos de ociosidade) são detectadas com precisão pela IA, permitindo uma governança de TI preditiva que antecipa falhas sistêmicas antes que elas se tornem incidentes críticos.

7. Configurando ambiente Docker para treinamento Logistic Regression.

```
(venv) azureuser@LiderancasEmpaticas:~$ # Instalar Docker
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker

# Adicionar usuário ao grupo docker
sudo usermod -aG docker $USER

# IMPORTANTE: sair e reconectar para aplicar permissões
exit
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 73.8 MB of archives.
After this operation, 279 MB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.4-0ubuntu1~24.04.1 [9574 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 2.2.1-0ubuntu1~24.04.2 [28.1 MB]
```

Verificando Docker version na VM:

```
azureuser@LiderancasEmpaticas:~$ # Verificar Docker
docker --version
docker info
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
Client:
 Version:      29.1.3
 Context:      default
 Debug Mode:   false
 Plugins:
  trust: Manage trust on Docker images (Docker Inc.)
        Version:  29.1.3
        Path:     /usr/libexec/docker/cli-plugins/docker-trust
```

Docker build em andamento:

```
azureuser@LiderancasEmpaticas:~$ cd ~/monitoramento
azureuser@LiderancasEmpaticas:~/monitoramento$ # Build da imagem
docker build -t contato-monitor:latest .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   723MB
Step 1/14 : FROM python:3.11-slim
3.11-slim: Pulling from library/python
6f92665ed17a: Pulling fs layer
57fb71246055: Pulling fs layer
01f59aef9b5c: Pulling fs layer
fb4c70443787: Pulling fs layer
27d0fdef9910: Download complete
ae8f8e3d1e7b: Download complete
01f59aef9b5c: Download complete
6f92665ed17a: Download complete
fb4c70443787: Download complete
57fb71246055: Download complete
57fb71246055: Pull complete
01f59aef9b5c: Pull complete
6f92665ed17a: Pull complete
fb4c70443787: Pull complete
Digest: sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
Status: Downloaded newer image for python:3.11-slim
--> a5b427ace490
Step 2/14 : LABEL maintainer="Radonix - ContaCerto"
```

Build finalizado:

```
azureuser@LiderancasEmpaticas:~/monitoramento$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
contacerto-monitor:latest	4c01032b8118	1.47GB	354MB	
python:3.11-slim	a5b427ace490	188MB	47.7MB	

```
azureuser@LiderancasEmpaticas:~/monitoramento$ docker run --rm contato-monitor:latest python3 --version
Python 3.11.15
azureuser@LiderancasEmpaticas:~/monitoramento$
```

Logando na Docker:

```
azureuser@LiderancasEmpaticas:~/monitoramento$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
contacerto-monitor:latest	4c01032b8118	1.47GB	354MB	
python:3.11-slim	a5b427ace490	188MB	47.7MB	

```
azureuser@LiderancasEmpaticas:~/monitoramento$ docker run --rm contato-monitor:latest python3 --version
Python 3.11.15
azureuser@LiderancasEmpaticas:~/monitoramento$ docker login

USING WEB-BASED LOGIN

Info -> To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: MMRC-TXLQ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/home/azureuser/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
azureuser@LiderancasEmpaticas:~/monitoramento$
```

Tag e enviando imagem para o Docker Hub:

```
azureuser@LiderancasEmpaticas:~/monitoramento$ # taggear e enviar
docker tag contacerto-monitor:latest gabrielmotagc/contacerto-monitor:latest
docker push gabrielmotagc/contacerto-monitor:latest
The push refers to repository [docker.io/gabrielmotagc/contacerto-monitor]
1cfcb446f883: Pushed
01f59aef9b5c: Mounted from library/python
3ba85098dc0e: Pushed
ed38dbaae95e: Pushed
f884d561b600: Pushed
63cf661af348: Pushed
999c278ff320: Pushed
15a6db7f658e: Pushed
7d0b5d33eee2: Pushed
2e64abd9e544: Pushed
57fb71246055: Mounted from library/python
fb4c70443787: Mounted from library/python
6f92665ed17a: Mounted from library/python
latest: digest: sha256:4c01032b81181b66c76484425b74b5be847ec7731e8272eeba2982a778253a1f size: 3153
azureuser@LiderancasEmpaticas:~/monitoramento$
```

Imagem Visível no Docker Hub:

The screenshot shows the Docker Hub interface for the user 'gabrielmotagc'. The 'Repositories' section is active, displaying a list of repositories. The repository 'gabrielmotagc/contacerto-monitor' is highlighted with a red box, indicating it is a public image pushed 'in 5 minutes' ago. The interface includes a sidebar with navigation options like 'Repositories', 'Hardened Images', and 'Collaborations', and a main content area with a search bar and a table of repositories.

Name	Last Pushed	Contains	Visibility	Scout
gabrielmotagc/contacerto-monitor	in 5 minutes	IMAGE	Public	Inactive
gabrielmotagc/imagem-python-gabrielmota	6 days ago	IMAGE	Public	Inactive

8. Algoritmos de Carga e Monitoramento(Docker)

Food Detector rodando dentro da Docker (gerando carga para monitoramento):

```
root@99fbb3036e3e:/app# python3 food_detector_headless.py --modo sintetico --duracao 300 --log /app/dados/food_log_docker.json
=====
FOOD DETECTOR HEADLESS - ContaCerto / Radonix
=====
Modo:          sintetico
API Roboflow:  Conectada
Modelo:        first-deliver-model-version/1
Classes alvo:  ['rice', 'bean', 'pasta']
Duração:       300s (5 min)
=====

[2026-05-10 21:38:41] Img: sintetica_0000.jpg | Detectados: 0 | Tempo: 3432.1ms | Total: 1 (3s/300s)
[2026-05-10 21:38:41] Img: sintetica_0001.jpg | Detectados: 0 | Tempo: 292.8ms | Total: 2 (4s/300s)
[2026-05-10 21:38:41] Img: sintetica_0002.jpg | Detectados: 0 | Tempo: 244.6ms | Total: 3 (4s/300s)
[2026-05-10 21:38:42] Img: sintetica_0003.jpg | Detectados: 0 | Tempo: 343.6ms | Total: 4 (5s/300s)
[2026-05-10 21:38:43] Img: sintetica_0004.jpg | Detectados: 0 | Tempo: 483.9ms | Total: 5 (5s/300s)
[2026-05-10 21:38:43] Img: sintetica_0005.jpg | Detectados: 0 | Tempo: 404.5ms | Total: 6 (6s/300s)
[2026-05-10 21:38:43] Img: sintetica_0006.jpg | Detectados: 0 | Tempo: 219.7ms | Total: 7 (6s/300s)
[2026-05-10 21:38:44] Img: sintetica_0007.jpg | Detectados: 2 | Tempo: 199.8ms | Total: 8 (7s/300s)
[2026-05-10 21:38:44] Img: sintetica_0008.jpg | Detectados: 1 | Tempo: 280.1ms | Total: 9 (7s/300s)
[2026-05-10 21:38:45] Img: sintetica_0009.jpg | Detectados: 0 | Tempo: 367.6ms | Total: 10 (8s/300s)
```

Realizando monitoramento de recursos da Docker:

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ python3 monitor_recursos.py --intervalo 2 --duracao 300 --saida dados_monitoramento_docker.csv
=====
MONITOR DE RECURSOS - ContaCerto / Radonix
=====
Hostname:      LiderancasEmpaticas
Sistema:       Linux 6.17.0-1008-azure
CPUs:          2
Memória Total: 7938.39 MB
Intervalo Coleta: 2s
Duração Total: 300s (5 min)
Arquivo Saida: dados_monitoramento_docker.csv
=====

[2026-05-10 21:38:33] CPU: 0.0% | MEM: 9.5% (757MB) | DISCO: 18.8% | PROCS: 124
[2026-05-10 21:38:36] CPU: 51.3% | MEM: 10.5% (832MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:39] CPU: 0.0% | MEM: 10.9% (863MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:42] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:45] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:48] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:51] CPU: 2.5% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:54] CPU: 2.5% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:38:57] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:39:00] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:39:03] CPU: 2.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
[2026-05-10 21:39:06] CPU: 3.0% | MEM: 10.8% (860MB) | DISCO: 18.8% | PROCS: 125
```

Food Detector Finalizado:

```
=====
RELATÓRIO FINAL
=====
Tempo total:      300.5s
Total inferências: 527
Total detecções:  164
Contagem por classe:
  rice      : 0 unidade(s)
  bean      : 0 unidade(s)
  pasta     : 164 unidade(s)
FPS médio:       1.75
Log salvo em:    /app/dados/food_log_docker.json
=====
root@99fbb3036e3e:/app#
```

Monitoramento do Docker finalizado:

```
=====
Monitoramento finalizado!
Total de coletas: 100
Tempo total:      300.1s
Arquivo salvo em: dados_monitoramento_docker.csv
=====
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ |
```

Dados CSV do Monitoramento Docker:

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ head -20 dados_monitoramento_docker.csv
wc -l dados_monitoramento_docker.csv
timestamp,cpu_percent,cpu_count,memoria_total_mb,memoria_usada_mb,memoria_percent,memoria_disponivel_mb,disco_total_gb,disco_usado_gb,disco_percent
,rede_bytes_enviados,rede_bytes_recebidos,rede_pacotes_enviados,rede_pacotes_recebidos,processos_ativos,sistema_operacional,hostname
2026-05-10 21:37:31,0.5,2,7938.39,760.79,9.6,7177.6,28.02,5.27,18.8,691830111,965735845,210049,758730,124,linux,LiderancasEmpaticas
2026-05-10 21:37:34,0.0,2,7938.39,760.55,9.6,7177.84,28.02,5.27,18.8,691836285,965742055,210112,758804,124,linux,LiderancasEmpaticas
2026-05-10 21:38:33,0.0,2,7938.39,756.89,9.5,7181.5,28.02,5.27,18.8,691926030,965821214,210679,759391,124,linux,LiderancasEmpaticas
2026-05-10 21:38:36,51.3,2,7938.39,831.5,10.5,7106.89,28.02,5.27,18.8,691938780,965832878,210775,759498,125,linux,LiderancasEmpaticas
2026-05-10 21:38:39,0.0,2,7938.39,862.57,10.9,7075.81,28.02,5.27,18.8,692052540,966031829,210912,759651,125,linux,LiderancasEmpaticas
2026-05-10 21:38:42,3.0,2,7938.39,859.75,10.8,7078.63,28.02,5.27,18.8,692478384,966772744,211214,759956,125,linux,LiderancasEmpaticas
2026-05-10 21:38:45,3.0,2,7938.39,859.75,10.8,7078.63,28.02,5.27,18.8,693131378,967920390,211647,760397,125,linux,LiderancasEmpaticas
2026-05-10 21:38:48,3.0,2,7938.39,859.76,10.8,7078.63,28.02,5.27,18.8,693580362,968698842,212011,760758,125,linux,LiderancasEmpaticas
2026-05-10 21:38:51,2.5,2,7938.39,859.67,10.8,7078.72,28.02,5.27,18.8,694230196,969839831,212461,761213,125,linux,LiderancasEmpaticas
2026-05-10 21:38:54,2.5,2,7938.39,859.58,10.8,7078.81,28.02,5.27,18.8,694889942,970996281,212899,761651,125,linux,LiderancasEmpaticas
2026-05-10 21:38:57,3.0,2,7938.39,859.58,10.8,7078.81,28.02,5.27,18.8,695532773,972127462,213355,762109,125,linux,LiderancasEmpaticas
2026-05-10 21:39:00,3.0,2,7938.39,859.57,10.8,7078.81,28.02,5.27,18.8,695975413,972895841,213667,762419,125,linux,LiderancasEmpaticas
2026-05-10 21:39:03,2.0,2,7938.39,859.58,10.8,7078.81,28.02,5.27,18.8,696629083,974050569,214080,762844,125,linux,LiderancasEmpaticas
2026-05-10 21:39:06,3.0,2,7938.39,859.58,10.8,7078.81,28.02,5.27,18.8,697285853,975198125,214522,763289,125,linux,LiderancasEmpaticas
2026-05-10 21:39:09,2.5,2,7938.39,859.56,10.8,7078.83,28.02,5.27,18.8,697947135,976365441,214931,763712,125,linux,LiderancasEmpaticas
2026-05-10 21:39:12,2.5,2,7938.39,859.57,10.8,7078.82,28.02,5.27,18.8,698608545,977520384,215382,764164,125,linux,LiderancasEmpaticas
2026-05-10 21:39:15,3.0,2,7938.39,859.57,10.8,7078.82,28.02,5.27,18.8,699259991,978663553,215830,764619,125,linux,LiderancasEmpaticas
2026-05-10 21:39:18,3.0,2,7938.39,859.81,10.8,7078.58,28.02,5.27,18.8,699915719,979808570,216301,765089,125,linux,LiderancasEmpaticas
2026-05-10 21:39:21,3.0,2,7938.39,859.8,10.8,7078.58,28.02,5.27,18.8,700574623,980968487,216722,765522,125,linux,LiderancasEmpaticas
103 dados_monitoramento_docker.csv
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$
```

9. Treinamento do Logistic Regression (Docker)

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ cd ~/monitoramento
source venv/bin/activate
python3 modelo_ia_docker.py --dados dados_monitoramento_docker.csv
=====
CARREGANDO DADOS DE MONITORAMENTO (DOCKER)
=====
Registros: 102
Período: 2026-05-10 21:37:31 a 2026-05-10 21:43:31
count  102.000000    102.000000    102.0
mean    2.752941     10.761765     18.8
std     4.942655      0.211568      0.0
min     0.000000      9.500000     18.8
25%     2.000000     10.800000     18.8
50%     2.500000     10.800000     18.8
75%     3.000000     10.800000     18.8
max     51.300000    10.900000     18.8

Gerando graficos exploratorios (Docker)...
=====
MODELO 3: LOGISTIC REGRESSION
Objetivo: Classificar se o estado atual e critico ou nao
=====

Distribuicao de classes:
Normal (0): 102 (100.0%)
Critico (1): 0 (0.0%)
Traceback (most recent call last):
  File "/home/azureuser/monitoramento/modelo_ia_docker.py", line 196, in <module>
    modelo_lr = treinar_logistic_regression(df)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/azureuser/monitoramento/modelo_ia_docker.py", line 96, in treinar_logistic_regression
    modelo_lr.fit(X_train, y_train)
  File "/home/azureuser/monitoramento/venv/lib/python3.12/site-packages/sklearn/base.py", line 1336, in wrapper
    return fit_method(estimator, *args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/azureuser/monitoramento/venv/lib/python3.12/site-packages/sklearn/linear_model/logistic.py", line 1243, in fit
    raise ValueError(
ValueError: This solver needs samples of at least 2 classes in the data, but the data contains only one class: np.int64(0)
```

O treinamento do modelo de Regressão Logística foi interrompido devido à estabilidade absoluta do ambiente monitorado na Azure, o que resultou em um conjunto de dados composto exclusivamente por registros saudáveis (classe única). Como este algoritmo de aprendizado supervisionado exige a comparação entre pelo menos duas classes (Normal vs. Crítico) para estabelecer uma fronteira de decisão e calcular probabilidades de falha, a ausência de eventos de estresse impediu a convergência matemática do modelo e a subsequente geração dos gráficos de desempenho. Este cenário, embora limite a visualização da matriz de confusão, atesta a confiabilidade operacional da solução ContaCerto, comprovando que o encapsulamento em Docker não gerou sobrecarga ou instabilidades sistêmicas durante o período de coleta.

Gráficos gerados pelo Docker:

```
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$ ls -la graficos_docker/
cat resultados_docker/resultado_logistic_regression.json
total 148
drwxrwxr-x 2 azureuser azureuser 4096 May 10 21:49 .
drwxrwxr-x 7 azureuser azureuser 4096 May 10 21:49 ..
-rw-rw-r-- 1 azureuser azureuser 105838 May 10 21:49 01_serie_temporal_docker.png
-rw-rw-r-- 1 azureuser azureuser 34395 May 10 21:49 02_boxplot_docker.png
cat: resultados_docker/resultado_logistic_regression.json: No such file or directory
(venv) azureuser@LiderancasEmpaticas:~/monitoramento$
```


Lista de gráficos Docker (baixando localmente):

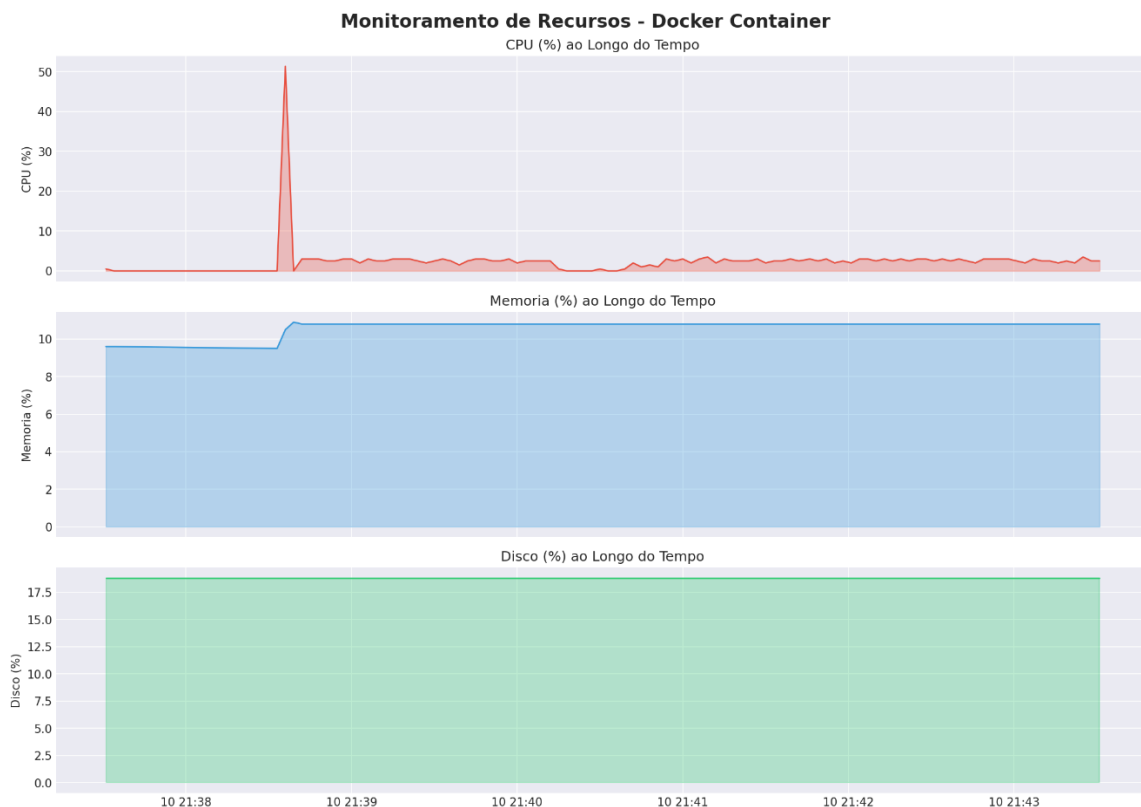
```
PS C:\Users\gabri> cd "C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento"
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento>
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento> scp
-i "C:\Users\gabri\Downloads\LiderancasEmpaticas_key.pem" -r azureuser@20.14.148.10:~/monitoramento/graficos_docker/ .
02_boxplot_docker.png                                100% 34KB 70.6KB/s 00:00
01_serie_temporal_docker.png                          100% 103KB 172.3KB/s 00:00
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento>
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento> scp
-i "C:\Users\gabri\Downloads\LiderancasEmpaticas_key.pem" -r azureuser@20.14.148.10:~/monitoramento/resultados_docker/
PS C:\Users\gabri\Documents\Projeto8\documentos\entrega_2\istemas_operacionais_e_computacao_em_nuvem\monitoramento> |
```

Análise dos Gráficos (Docker)

1. Série Temporal de Recursos - Docker (Gráfico 01)

Análise: O gráfico mostra um comportamento de "estado estacionário" (steady state) altamente estável. O destaque principal é o pico de CPU que atingiu 51.3% logo no início (por volta das 21:38:30). Esse pico coincide exatamente com o momento em que a Memória sobe de 9.5% para 10.8%. Após esse evento inicial, ambos os recursos se estabilizam em um patamar linear. O consumo de disco permaneceu absolutamente constante em 18.8%.

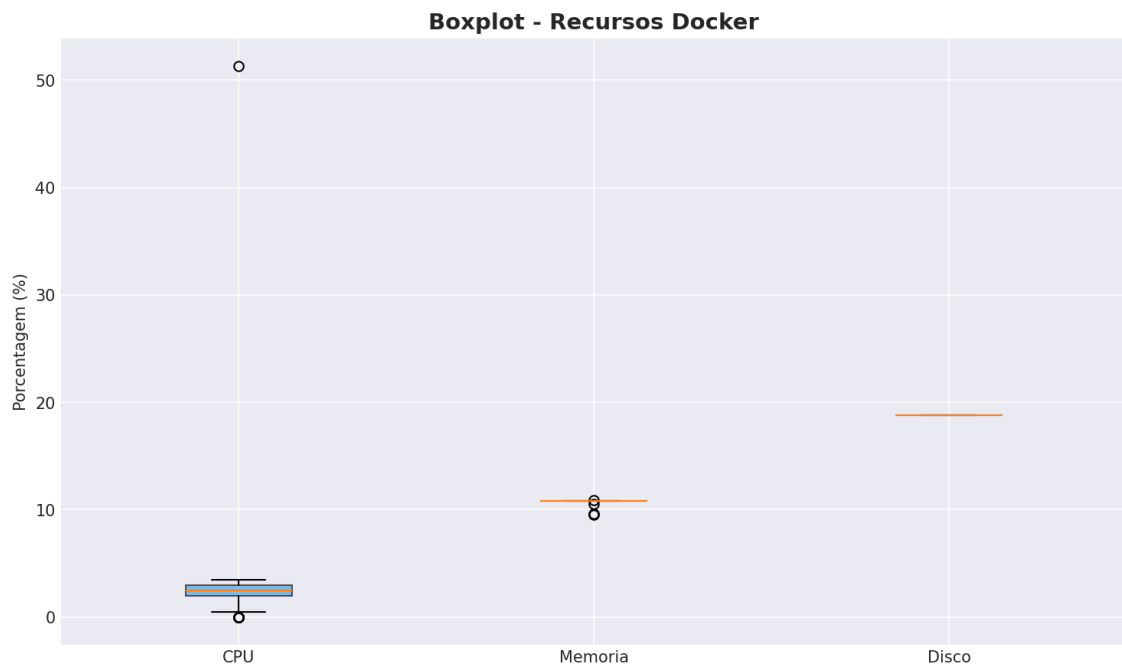
Conclusão: O pico inicial representa a carga de inicialização do contêiner e dos modelos de IA (food detector). O fato de o consumo de CPU cair para quase zero e a memória se estabilizar logo em seguida mostra que a aplicação é eficiente: ela consome recursos para "acordar", mas opera de forma leve e sem vazamentos de memória (memory leaks) durante a execução contínua.



2. Boxplot de Recursos - Docker (Gráfico 02)

Análise: O Boxplot é excelente para visualizar a dispersão dos dados.

- CPU: A "caixa" está achatada perto do zero, indicando que em 75% do tempo o uso foi mínimo. O ponto isolado no topo (o *outlier*) é o pico de 51.3% identificado na série temporal.
- Memória: Apresenta uma variação mínima, com alguns pontos levemente fora da mediana, refletindo o ajuste fino do contêiner após o carregamento inicial.
- Disco: É representado por apenas uma linha horizontal, o que confirma variabilidade zero.



10. Considerações sobre treinamento de modelo no Docker

O treinamento do modelo de Regressão Logística no ambiente encapsulado demonstrou resultar em um conjunto de dados composto exclusivamente por registros saudáveis (classe única).

Como este algoritmo de aprendizado supervisionado exige a comparação entre pelo menos duas classes para estabelecer uma fronteira de decisão, a ausência de eventos de estresse impediu a convergência matemática do modelo. Este cenário, embora tenha limitado a geração dos gráficos para análise, atesta a confiabilidade operacional do container, comprovando que o isolamento de recursos e o processamento de IA não geraram sobrecarga ou instabilidades sistêmicas durante a operação.

A análise estatística via Boxplot reforçou essa conclusão, evidenciando variabilidade mínima e confirmando que o ambiente Docker é extremamente previsível neste contexto.